

Smart Parking System

Software and Programming_
in IoT

Done by:
Darshini P-2201102015
Shruthi MS -2201102051



An isometric illustration of a smart city parking lot. The scene features a dark blue road with white lane markings. On the left, there's a green grassy area with a yellow bench and a small green tree. A blue signpost with a white 'LOT' sign stands near the road. Several tall, thin poles with circular bases and white Wi-Fi-like signal icons are positioned around the area, representing a wireless sensor network. A blue car is parked in a designated spot. Two stylized human figures are standing on the road. In the background, there are blue buildings and a blue sky with a yellow sun and a blue cloud with a gear icon. The overall color palette is dominated by blues, greens, and yellows.

Introduction

- In smart cities, there is a greater need for new and effective technology to tackle many of the problems that are visible on the surface, as well as to make cities less crowded.
- Finding a parking spot is one of the most aggravating issues for drivers. Particularly in public venues such as shopping malls, 5-star hotels, and multiplex cinema halls. They concentrate on practical smart parking technologies developed to address existing issues through the use of a wireless sensor network and real-time data processing from the sensors.
- In this study, we create and design a smart parking system that effectively addresses these issues.

Objective

The objective of a smart parking system using IoT is to optimize and streamline the process of parking by leveraging interconnected devices and technologies.

The system aims to enhance the efficiency, convenience, and sustainability of parking operations by integrating sensors, communication networks, and data analytics.

Through IoT-enabled sensors and devices, the system can provide accurate information about available parking spaces, guide drivers to vacant spots using navigation systems, and enable remote monitoring and control of parking facilities

Proposed Methodology

Sensor Integration:

Infrared sensors were installed at entry and exit to detect the cars

Hardware Setup:

LCD displays parking availability, and a DHT11 sensor with a buzzer alerts for high temperatures

Testing & Deployment:

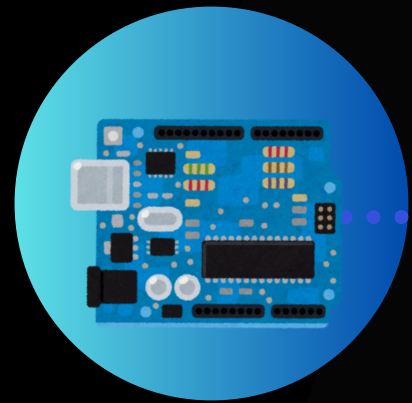
The system was rigorously tested with various scenarios to ensure sensor accuracy and system responsiveness.

Centralized Management:

A central system was developed to monitor parking availability, process data, and manage user requests.

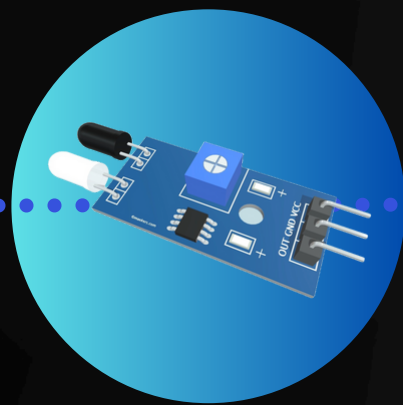


Key Components Used



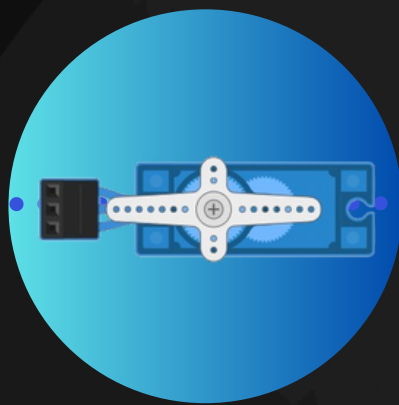
01

ARDUINO UNO



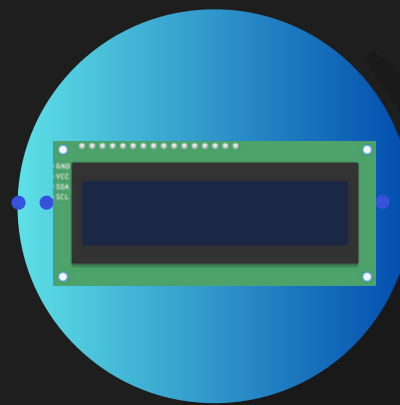
02

IR SENSOR



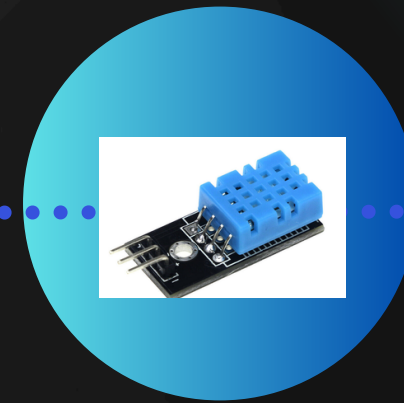
03

SERVO
MOTOR



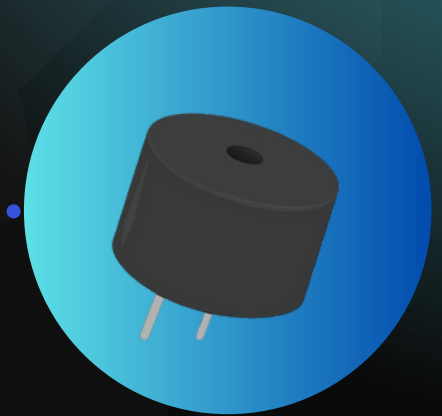
04

LCD DISPLAY



05

TEMPERATURE
SENSOR



06

BUZZER

Software Used

Arduino IDE with relevant libraries

Servo.h:

- PWM control functions
- Position control methods
- Timing management

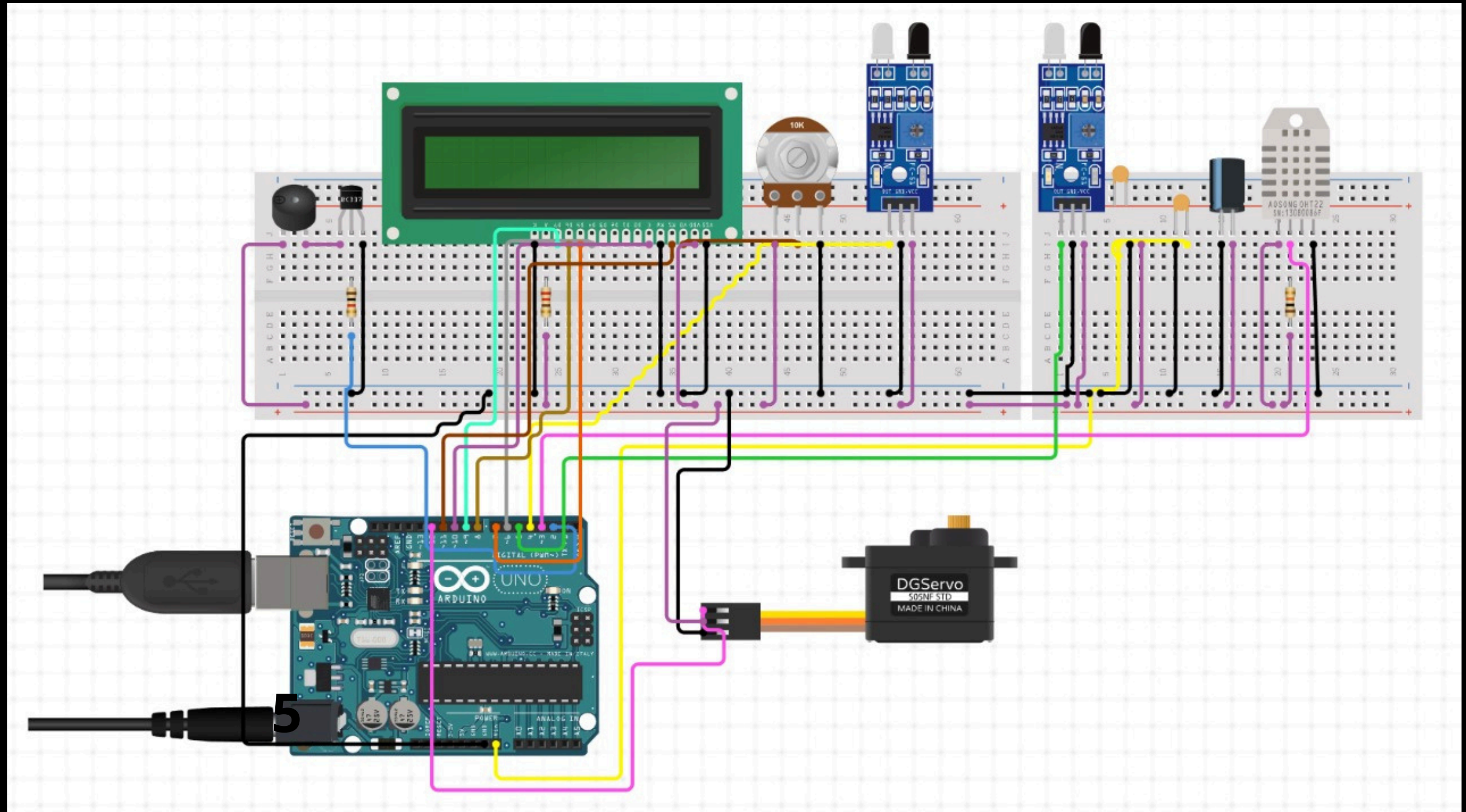
Liquid Crystal:

- test display
- cursor control
- backlight control
- custom characters

The DHT11 sensor

continuously measures the ambient temperature in the parking area. This can be useful for monitoring the environmental conditions in the parking space, which could impact things

Circuit Diagram



System WorkFlow

- 1 IR sensors detect vehicle presence.
 - 2 Data sent to Arduino Uno.
 - 3 Servo motor controls gate access.
 - 4 The DHT11 sensor monitors the parking area's temperature, and the fire alarm triggers if a fire is detected
 - 5 The LCD shows available parking slots and displays an evacuation message if a fire alarm is triggered
 - 6 The Arduino acts as the microcontroller, managing parking detection, LCD display, and fire alarm with the DHT11 sensor
- 

Code Walkthrough

```
1  #include <LiquidCrystal.h>
2  #include <Servo.h>
3  #include <DHT.h>
4
5  LiquidCrystal lcd(12, 11, 5, 4, 3, 2); // pins for lcd
6
7  Servo myservo1;
8
9  const int IR1 = 8; // Entry IR sensor
10 const int IR2 = 9; // Exit IR sensor
11
12 // Servo angles
13 const int servoOpen = 90;
14 const int servoClose = 0;
15
16 int Slot = 5;
17 const int maxSlots = 5; // Max slot capacity
18
19 // Flags to prevent multiple detections
20 bool entryDetected = false;
21 bool exitDetected = false;
22
23 #define DHTPIN 13 // Pin for temp sensorr
24 #define DHTTYPE DHT11
25 DHT dht(DHTPIN, DHTTYPE);
26
27 const int buzzerPin = 10; // pin for buzzer
28 const float tempThreshold = 30.0; // threshold temp
29
30 void setup() {
31     Serial.begin(9600); // Initializing serial communication
32
33     // Initializing LCD
34     analogWrite(6, 75); // contrast
35     lcd.begin(16, 2);
36
37     // Initializing IR sensor pins
38     pinMode(IR1, INPUT);
39     pinMode(IR2, INPUT);
40
41     // Initializing servo
42     myservo1.attach(7);
43     myservo1.write(servoClose);
44
45     // Initializing buzzer
46     pinMode(buzzerPin, OUTPUT);
47     digitalWrite(buzzerPin, LOW);
48
49     // Initializing DHT11
50     dht.begin();
```

```
51     displaySlots();
52 }
53
54 void loop() {
55     // Read IR sensor states
56     int sensor1 = digitalRead(IR1);
57     int sensor2 = digitalRead(IR2);
58
59     // Detect entry (sensor1 LOW)
60     if (sensor1 == LOW && !entryDetected) {
61         delay(50); // Debounce delay
62         if (digitalRead(IR1) == LOW) { // Confirm the sensor state
63             entryDetected = true;
64             handleEntry();
65         }
66     }
67
68     // Detect exit (sensor2 LOW)
69     if (sensor2 == LOW && !exitDetected) {
70         delay(50); // Debounce delay
71         if (digitalRead(IR2) == LOW) { // Confirm the sensor state
72             exitDetected = true;
73             handleExit();
74         }
75     }
76
77     // Reset entry flag when sensor1 is not triggered
78     if (sensor1 == HIGH && entryDetected) {
79         entryDetected = false;
80     }
81
82     // Reset exit flag when sensor2 is not triggered
83     if (sensor2 == HIGH && exitDetected) {
84         exitDetected = false;
85     }
86
87     // Check temp and activate buzzer if temp is too high
88     checkTemperature();
89
90     delay(100); // Small delay to avoid bouncing
91 }
92
93 // Function to handle car entry
94 void handleEntry() {
95     if (Slot > 0) {
96         // Open gate
97         myservo1.write(servoOpen);
98         Serial.println("Gate opened for entry.");
99         delay(2000); // Wait for gate to open and car to pass
100     }
```

```
101     // Close gate
102     myservo1.write(servoClose);
103     Serial.println("Gate closed after entry.");
104
105     // Update slot count
106     Slot--;
107     Serial.print("Slot decremented: ");
108     Serial.println(Slot);
109
110     // Update LCD
111     displaySlots();
112 } else {
113     // Parking full
114     lcd.clear();
115     lcd.setCursor(0, 0);
116     lcd.print(" SORRY :( ");
117     lcd.setCursor(0, 1);
118     lcd.print(" Parking Full ");
119     Serial.println("Parking Full");
120     delay(3000);
121     lcd.clear();
122
123     // Optionally, display welcome and slot count again
124     displaySlots();
125 }
126 }
127
128 // Function to handle car exit
129 void handleExit() {
130     if (Slot < maxSlots) {
131         // Open gate
132         myservo1.write(servoOpen);
133         Serial.println("Gate opened for exit.");
134         delay(2000); // Wait for gate to open and car to exit
135
136         // Close gate
137         myservo1.write(servoClose);
138         Serial.println("Gate closed after exit.");
139
140         // Update slot count
141         Slot++;
142         Serial.print("Slot incremented: ");
143         Serial.println(Slot);
144
145         // Update LCD
146         displaySlots();
147     }
148 }
149 }
```

Output Serial Monitor ✕

```
149
150 // Function to display available slots on LCD
151 void displaySlots() {
152     lcd.clear();
153     lcd.setCursor(0, 0);
154     lcd.print(" CAR PARKING ");
155     lcd.setCursor(0, 1);
156     lcd.print(" Slots Left: ");
157     lcd.print(Slot);
158     Serial.print(" Slots Left: ");
159     Serial.println(Slot);
160 }
161
162 // Function to check temperature and activate buzzer if too high
163 void checkTemperature() {
164     float temp = dht.readTemperature();
165     if (isnan(temp)) {
166         Serial.println("Failed to read temperature!");
167     } else {
168         Serial.print("Temperature: ");
169         Serial.print(temp);
170         Serial.println("°C");
171
172         // Check if temperature exceeds threshold
173         if (temp > tempThreshold) {
174             digitalWrite(buzzerPin, HIGH); // Turn on the buzzer
175
176             // Display warning message
177             Serial.println("FIRE DETECTED, EVACUATE QUICKLY AS POSSIBLE!");
178
179             lcd.clear();
180             lcd.setCursor(0, 0);
181             lcd.print(" FIRE DETECTED ");
182             lcd.setCursor(0, 1);
183             lcd.print(" EVACUATE NOW ");
184         } else {
185             digitalWrite(buzzerPin, LOW); // Turn off the buzzer
186             displaySlots(); // Reset LCD to display slot info
187         }
188     }
189 }
190 }
```

Hardware:

- IR Sensors: Detect entry/exit.
- Servo Motor: Opens/closes gate.
- LCD Display: Shows available slots.
- DHT11 Sensor & Buzzer: Fire alarm system.

Functions:

- Entry/Exit Handling: Updates parking slots, controls gate, prevents double-counting.
- Fire Detection: Alerts if temperature > 30°C.

Flow:

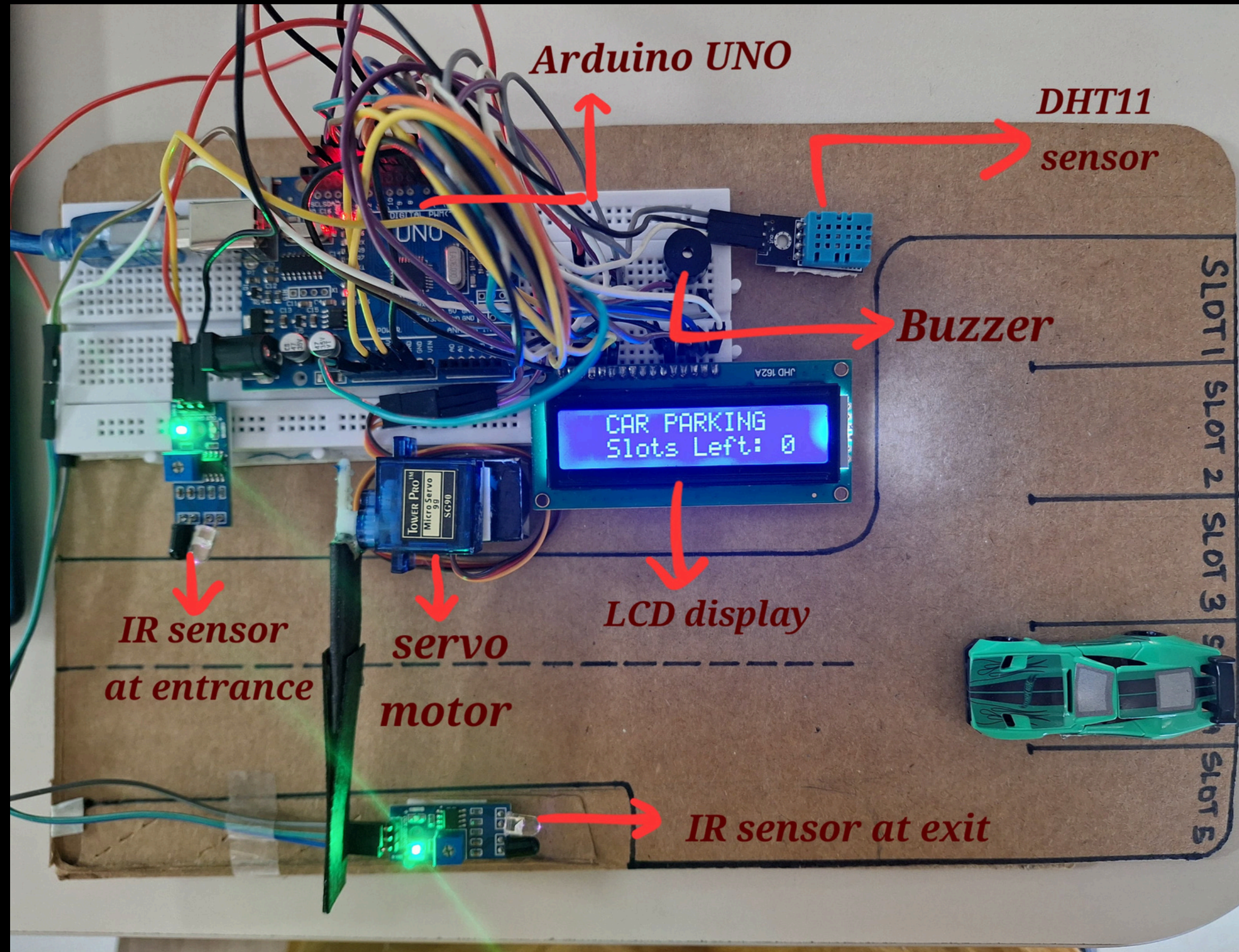
- Car detected → Gate opens/closes → Slot count updated.
- High temperature → Buzzer & fire warning on LCD.

Applications

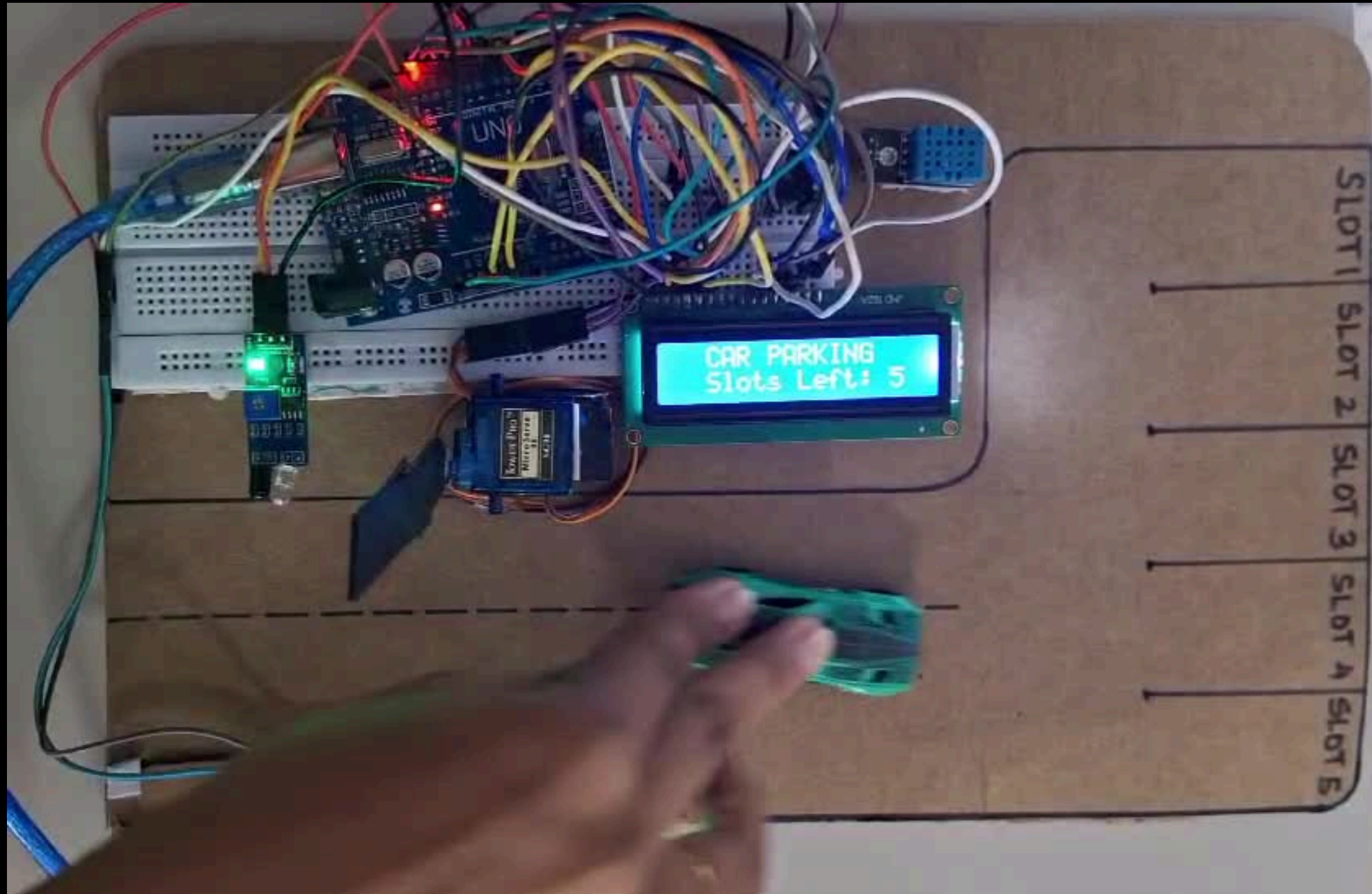
- Real-time Parking Space Monitoring: The system can utilize various sensors such as ultrasonic or infrared sensors to detect the availability of parking spaces in a parking lot. This information can be displayed on a user interface or mobile app, allowing drivers to quickly identify vacant spots.
- Automated Parking Guidance: The smart parking system can guide drivers to the nearest available parking spot through indicators or digital displays. This can reduce the time spent searching for parking, minimize traffic congestion, and enhance the overall parking experience.
- Reservation: The system can incorporate a reservation feature that allows users to book parking spaces in advance.
- Traffic Management: By monitoring the occupancy of parking spaces, the system can collect data on parking patterns and analyze it to optimize traffic flow within the parking lot.

Inference

- Reduced Search Time: Helps drivers find parking spots faster.
- Lower Emissions: By reducing idle time and unnecessary driving.
- Better Parking Management: Real-time data helps optimize space usage.



RESULT/OUTPUT



The system detects incoming and outgoing car from the entry and exit using IR Sensors . Once the parking Lot is full , the system doesn't allow further more car on entering the place until one or more car leaves the parking lot. If due to some situation the fire arises, the temperature sensor detects and makes a alarm sound and evacuate the parking lot.

<https://drive.google.com/file/d/194NjNamK-Sr-GVIRXVxEv2tqIYU5mHR/view>

Output Video Link

RESULT/OUTPUT

Output Serial Monitor X

Message (Enter to send message to 'Arduino')

```
Temperature: 24.80°C  
Slots Left: 5  
Gate opened for entry.  
Gate closed after entry.  
Slot decremented: 4  
Slots Left: 4  
Temperature: 24.80°C  
Slots Left: 4  
Temperature: 24.80°C  
Slots Left: 4  
Temperature: 24.80°C  
Slots Left: 4  
Gate opened for entry.  
Gate closed after entry.  
Slot decremented: 3  
Slots Left: 3  
Temperature: 24.80°C  
Slots Left: 3
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino')

```
Slots Left: 1  
Temperature: 27.60°C  
Slots Left: 1  
Gate opened for entry.  
Gate closed after entry.  
Slot decremented: 0  
Slots Left: 0  
Temperature: 27.60°C  
Slots Left: 0  
Temperature: 27.60°C  
Slots Left: 0  
Temperature: 27.60°C  
Slots Left: 0  
Parking Full
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino Uno' on 'COM6')

```
Temperature: 27.60°C  
Slots Left: 1  
Temperature: 29.80°C  
Slots Left: 1  
Temperature: 29.80°C  
Slots Left: 1  
Temperature: 30.20°C  
FIRE DETECTED, EVACUATE QUICKLY AS POSSIBLE!  
Temperature: 30.20°C  
FIRE DETECTED, EVACUATE QUICKLY AS POSSIBLE!  
Temperature: 30.20°C  
FIRE DETECTED, EVACUATE QUICKLY AS POSSIBLE!  
Temperature: 30.20°C  
FIRE DETECTED, EVACUATE QUICKLY AS POSSIBLE!  
Temperature: 30.20°C
```


Conclusion

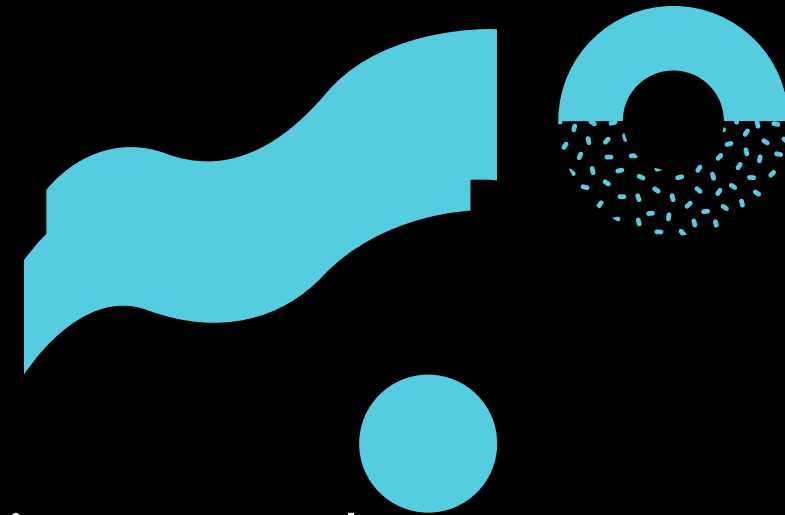
Key Takeaways:

- Our Smart Car Parking System provides a cost-effective and efficient solution to address modern parking challenges.
- It leverages real-time monitoring and automation using IoT technology to enhance the overall user experience.
- The system optimizes parking space utilization, reduces search time for drivers, and decreases congestion and emissions.

Benefits:

- Improved Efficiency: Helps drivers quickly locate available parking spots, reducing traffic and fuel consumption.
- User Convenience: Automated gate control and real-time slot availability display provide a seamless parking experience.
- Scalability: The system's modular design allows it to be easily scaled to accommodate larger parking areas.

Future Works



Impact:

These advancements will make the system more versatile, efficient, and aligned with future smart city developments. They will enhance the overall parking experience while supporting sustainability and urban efficiency.

Integration with Smart City IoT Systems:

Connect with other IoT-enabled systems like traffic lights, public transport, and smart navigation apps to provide a unified urban mobility solution. Share parking data with city management platforms for better traffic flow and resource planning.

Additional Features:

Predictive analysis using ARIMA (AutoRegressive Integrated Moving Average) is typically applied to time series data to forecast future values based on past trends. ARIMA to predict future parking slot availability based on historical data such as the number of available parking slots over time. And the available slots over the time using a mobile application

Future Works

To optimize parking availability, we are leveraging the ARIMA (AutoRegressive Integrated Moving Average) model for predictive analysis. ARIMA is a powerful statistical technique used for forecasting time series data. By analyzing historical data on parking slot occupancy (e.g., the number of free and occupied slots over different times of the day, days of the week, etc.), our model can identify patterns and trends to forecast future parking availability.

Key Benefits:

- **Accurate Forecasting:** The ARIMA model helps us predict the availability of parking spaces during different times of the day, enabling efficient parking space management.
- **Enhanced User Experience:** These predictions will be integrated into a mobile application that provides real-time updates to users, allowing them to plan ahead and find available parking slots more easily.
- **Dynamic Resource Allocation:** By understanding peak hours and low-demand periods, the system can optimize space usage and even support features like dynamic pricing to maximize revenue.

Current Progress:

We have developed an initial version of our machine learning model, which is currently in the early stages of testing and refinement. As we continue to gather more data and improve the model, we expect to achieve higher prediction accuracy, leading to a more robust and reliable parking management system.

Future Works–Current Progress

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split, GridSearchCV
4 from sklearn.tree import DecisionTreeRegressor
5 from sklearn.metrics import mean_squared_error, r2_score
6 import matplotlib.pyplot as plt
7 from statsmodels.tsa.arima.model import ARIMA
8 from datetime import datetime, timedelta
9 import statsmodels.api as sm
10
11 # Load historical parking data
12 csv_file = 'parking_data.csv' # Update this with your actual CSV file path
13 data = pd.read_csv(csv_file)
14
15 # Ensure Timestamp is a datetime object
16 data['Timestamp'] = pd.to_datetime([data['Timestamp']])
17
18 # Feature Engineering: Extract useful features from timestamp
19 data['Hour'] = data['Timestamp'].dt.hour
20 data['DayOfWeek'] = data['Timestamp'].dt.dayofweek
21 data['Day'] = data['Timestamp'].dt.day
22 data['Month'] = data['Timestamp'].dt.month
23
24 # Adding Rolling Averages as features (e.g., average parking slots over the past 3 hours)
25 data['RollingAvg'] = data['Available Slots'].rolling(window=3).mean().shift(1)
26
27 # Drop NaN values from rolling averages (first 3 rows will have NaN values)
28 data = data.dropna()
29
30 # Define features (X) and target (y)
31 X = data[['Hour', 'DayOfWeek', 'Day', 'Month', 'RollingAvg']] # Added Rolling Average
32 y = data['Available Slots'] # Target variable
33
34 # Split the data into training and testing sets
35 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
36
37 # Hyperparameter Tuning with GridSearchCV for Decision Tree
38 param_grid = {
39     'max_depth': [3, 5, 7, 10],
40     'min_samples_split': [2, 5, 10],
41     'min_samples_leaf': [1, 2, 4],
42 }
43 grid_search = GridSearchCV(DecisionTreeRegressor(random_state=42), param_grid, cv=5, n_jobs=-1)
44 grid_search.fit(X_train, y_train)
45
46 # Best model from grid search
47 best_model = grid_search.best_estimator_
48
49 # Make predictions using the best model
50 y_pred = best_model.predict(X_test)
51
52 # Evaluate the model
53 mse = mean_squared_error(y_test, y_pred)
54 r2 = r2_score(y_test, y_pred)
55 print(f"Best Model Parameters: {grid_search.best_params_}")
56 print(f"Mean Squared Error: {mse:.2f}")
```

```
56 print(f"Mean Squared Error: {mse:.2f}")
57 print(f"R^2 Score: {r2:.2f}")
58
59 # Visualize the actual vs predicted values
60 plt.figure(figsize=(10, 6))
61 plt.plot(y_test.values, label='Actual Slots', marker='o')
62 plt.plot(y_pred, label='Predicted Slots', marker='x')
63 plt.title('Actual vs Predicted Parking Slots')
64 plt.xlabel('Sample')
65 plt.ylabel('Available Slots')
66 plt.legend()
67 plt.grid(True)
68 plt.show()
69
70 # ARIMA model for time series forecasting
71 # Using the 'Available Slots' as the time series data
72 time_series = data['Available Slots']
73
74 # Split into train and test sets
75 train_size = int(len(time_series) * 0.8)
76 train, test = time_series[:train_size], time_series[train_size:]
77
78 # Fit ARIMA model
79 model = ARIMA(train, order=(5, 1, 0)) # (p, d, q) parameters
80 model_fit = model.fit()
81
82 # Forecast the next steps
83 forecast_steps = 10
84 forecast = model_fit.forecast(steps=forecast_steps)
85
86 # Plot the forecast
87 plt.figure(figsize=(10, 6))
88 plt.plot(time_series, label='Historical Data')
89 plt.plot(np.arange(len(time_series), len(time_series) + forecast_steps), forecast, label='Forecast', color='red')
90 plt.title('ARIMA Forecast - Parking Slot Availability')
91 plt.xlabel('Time')
92 plt.ylabel('Available Slots')
93 plt.legend()
94 plt.show()
95
96 # Future Prediction Function using Decision Tree or ARIMA
97 def predict_future_slots(hours_ahead=5):
98     future_dates = [datetime.now() + timedelta(hours=i) for i in range(1, hours_ahead + 1)]
99     future_df = pd.DataFrame({
100         'Hour': [date.hour for date in future_dates],
101         'DayOfWeek': [date.dayofweek for date in future_dates],
102         'Day': [date.day for date in future_dates],
103         'Month': [date.month for date in future_dates],
104         'RollingAvg': [data['Available Slots'].iloc[-1]] * hours_ahead # Use the last known available slot as average
105     })
106
107     # Make future predictions using the best Decision Tree model
108     future_predictions = best_model.predict(future_df)
109     return future_dates, future_predictions
110
111 # Predict for the next 5 hours
112 future_dates, future_predictions = predict_future_slots(hours_ahead=5)
113
114 # Display the future predictions
115 print("\nFuture Predictions (Next 5 Hours):")
116 for date, slots in zip(future_dates, future_predictions):
117     print(f"{date.strftime('%Y-%m-%d %H:%M:%S')} - Predicted Slots: {int(slots)}")
118
```

```
1 import dash
2 from dash import dcc, html
3 import pandas as pd
4 from dash.dependencies import Input, Output
5
6 app = dash.Dash(__name__)
7 csv_file = 'parking_data.csv'
8
9 app.layout = html.Div([
10     html.H1('Parking Availability Dashboard'),
11     dcc.Interval(id='interval-component', interval=10000, n_intervals=0),
12     dcc.Graph(id='live-update-graph')
13 ])
14
15 @app.callback(
16     Output('live-update-graph', 'figure'),
17     [Input('interval-component', 'n_intervals')]
18 )
19 def update_graph(n):
20     df = pd.read_csv(csv_file, names=['Timestamp', 'Slots'])
21     df['Timestamp'] = pd.to_datetime(df['Timestamp'])
22
23     fig = {
24         'data': [{'x': df['Timestamp'], 'y': df['Slots'], 'type': 'line'}],
25         'layout': {'title': 'Real-Time Parking Slot Availability'}
26     }
27     return fig
28
29 if __name__ == '__main__':
30     app.run_server(debug=True)
```

- Feature Engineering: Extracted features like Hour, DayOfWeek, and added rolling averages to improve model accuracy.
- Decision Tree Regression: Optimized using GridSearchCV for hyperparameter tuning, achieving strong performance with MSE and R² evaluation.
- ARIMA Time Series Forecasting: Forecasted future parking availability using (5, 1, 0) order, visualizing historical and predicted data.
- Mobile Integration: Predictions integrated into a mobile app for real-time parking slot updates.
- Future Predictions: Implemented a function to forecast parking slots for the next 5 hours using Decision Tree.
- Interactive Dashboard:
- Automatically updates every 10 seconds with new data from a CSV file. Provides users with up-to-date information on parking slots.

References

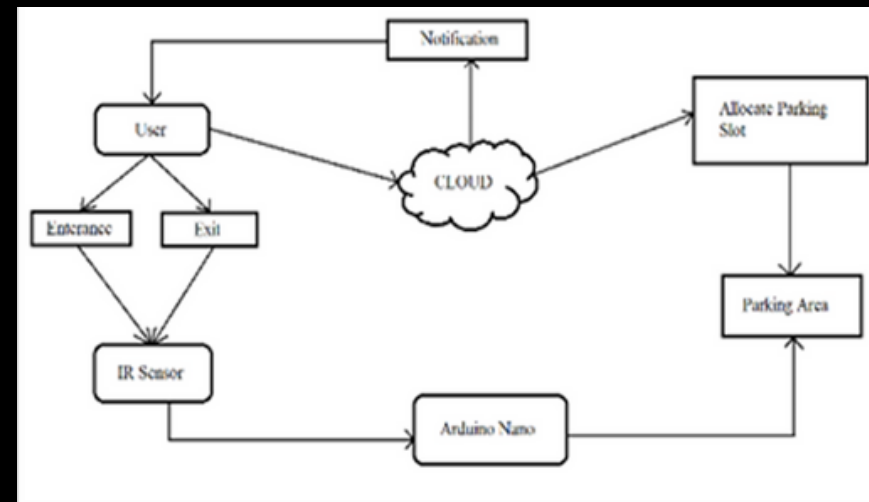
<https://www.ijeat.org/wp-content/uploads/papers/v9i1/A1963109119.pdf>

<https://ijrar.org/papers/IJRAR22C1478.pdf>

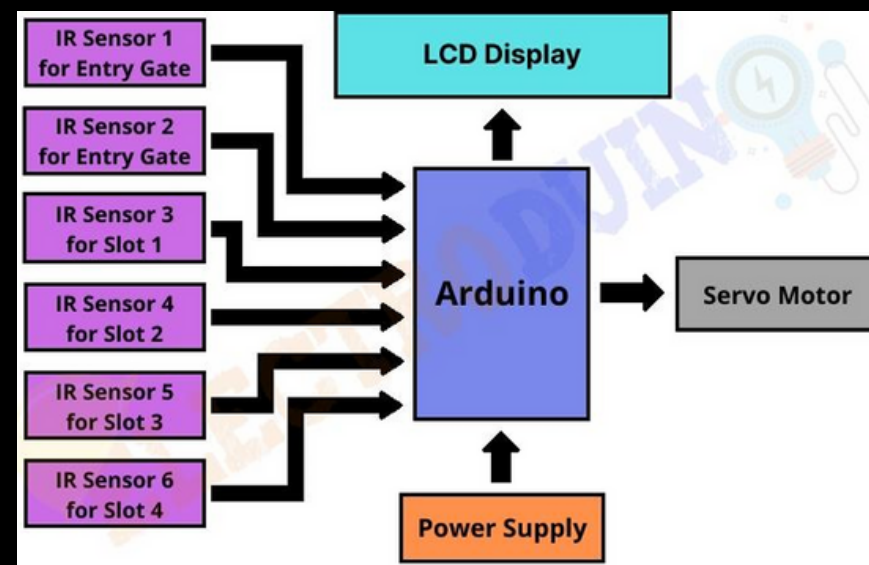
[Smart parking systems: comprehensive review based on various aspects – ScienceDirect](#)

https://www.irjmets.com/uploadedfiles/paper/issue_5_may_2022/24355/final/final_irjmets1653791250.pdf

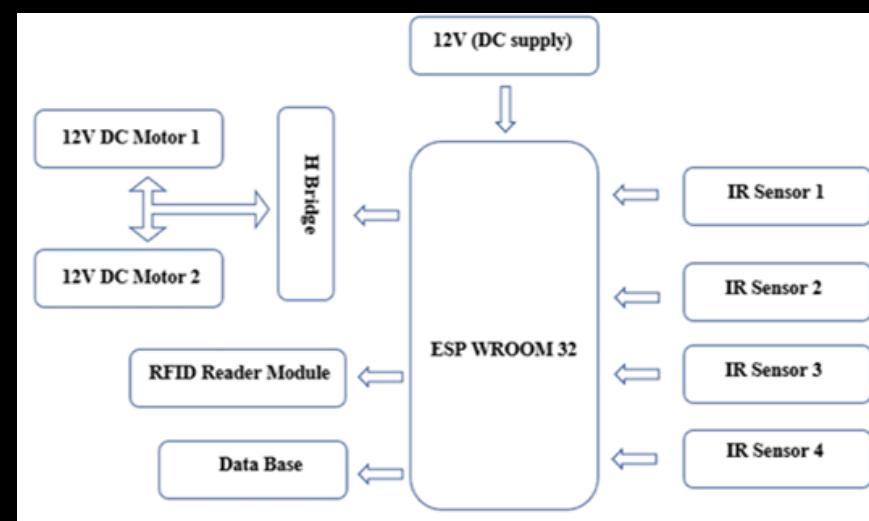
References- Comparative Analysis



1



2



3

Aspect	International Journal of Engineering and Advanced Technology (IJEAT) ISSN: 2249-8958 (Online), Volume-9 Issue-1, October, 2019 Smart Parking System using IoT 1	International Research Journal of Modernization in Engineering Technology and Science (Peer-Reviewed, Open Access, Fully Refereed International Journal) 2 Volume:04/Issue:05/May-2022 Impact Factor- 6.752	SMART PARKING SYSTEM USING IOT-IJRAR 3
Technology	Arduino, RFID, GSM, IR Sensors, Cloud	IoT (IR Sensors, RFID, Mobile App)	ESP WROOM 32, IR Sensors, RFID, Web Platform
Features	Real-time parking slot detection, SMS notifications, RFID, Cloud Integration	Slot booking, Dynamic pricing, LED indicators, Mobile App, Online Payment, Background subtraction with OpenCV	Early booking, Real-time updates, LCD displays, RFID, Web-based platform
Objective	Minimize congestion, reduce fuel consumption, prevent car theft	Optimize space usage, reduce emissions, enhance convenience	Reduce search time, improve traffic flow, reduce fuel consumption
System Focus	Sensor-based detection, Cloud integration	Mobile interaction, Real-time notifications, Payment systems	Web-based booking, Automated entry/exit
Microcontroller	Arduino	Arduino Uno	ESP WROOM 32
Sensors Used	IR Sensors	IR Proximity Sensors	IR Sensors
Communication Module	GSM, RFID	RFID, Mobile App	RFID, Web Interface
Cloud Integration	Yes	Yes	Yes
User Notification	SMS Notifications	Mobile App Notifications	Web Notifications
Slot Booking	Not Available	Available	Available
Payment Method	Not Specified	RFID, Online Payment	RFID, Online Payment
Security Features	RFID for Car Theft Prevention	License Plate Recognition	RFID Authentication
Strengths	- Car theft prevention using RFID - SMS notifications - Cloud integration	- Mobile app integration with real-time updates - Dynamic pricing - Visual indicators (LED) - Enhanced detection with OpenCV	- Web-based interface - Advanced microcontroller (ESP WROOM 32) - Automated slot
Limitations	- No mobile app for booking - No real-time slot booking - Limited user-friendly features	- Higher complexity due to multiple features (OpenCV, dynamic pricing) - Higher initial investment	- Web-based interface may not be as accessible as mobile apps - Basic slot management, no dynamic pricing
Recommendation	Ideal for simpler implementations with basic features	Best for projects aiming to enhance user experience through mobile apps and dynamic pricing	Best for environments where web-based interfaces are preferred